INDIAN INSTITUTE OF SCIENCE EDUCATION AND RESEARCH THIRUVANANTHAPURAM

(**An Autonomous Institution under MOE, Govt. of India**)

Maruthamala. P.O, Vithura, Thiruvananthapuram – 695551

School of Data Science

Computer Organization & Operating Systems

Mid-Semester Examination, Vasanth 2026

Maximum Marks: **80** Course Code: **DSC315**

Weightage: **30** Date: **2026-Feb-21**

Instructions:

- In Part A, Question 1 is compulsory. Select **one** question from Part B and **three** questions from Part C.
- Write all answers related to a single question in a contiguous location.

. . . . . .

Part-A

1.
   - Take your roll number, for example IMS12345, and note the last three digits: 345, say x.
   - Multiply the last digit by 0.0625 and add it to the number formed by the first two digits. Let the result be a.
   - Rearrange the three digits by placing the last digit in the front, say it y, then repeat the same calculation. Let the result be b.
   - Example, if roll= IMS12345, then $x = 345$, $y = 534$: and
     - $a = 34 + (5 \times 0.0625) = 34 + 0.3125 = 34.3125$
     - $b = 53 + (4 \times 0.0625) = 35 + 0.25 = 35.25$

   (a) Find 2's complement representations $-x$ and $-y$ in 16 bit format

   (b) Convert a and b to their binary representations.

   (c) Compute their product $c = a \times b$ using standard binary multiplication.

   (d) Determine the rounded value of a, b, c if only 3 bits are kept after the binary point. Give reasons in short.

   (e) Find 32-bit floating point representations of a and b, denoted by a' and b', resp.

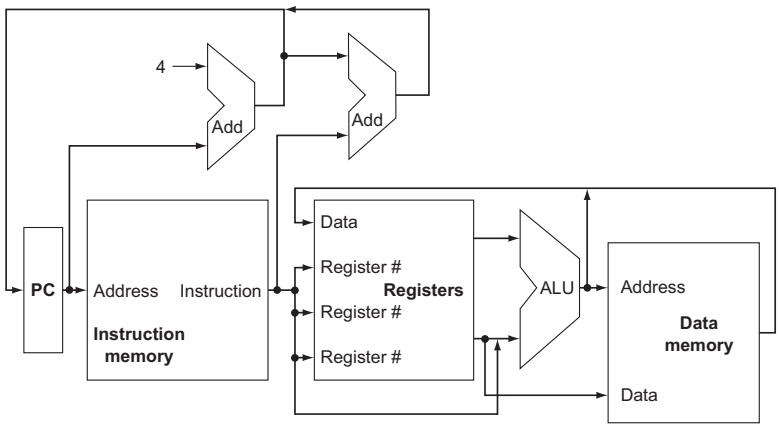
   (f) Step by step, compute $c' = a' + b'$ using floating point arithmetic.

   (g) Give the hexadecimal representation of c'.

   [2+2+4+4+6+10+2 =30]

Part-B

2. Consider the following abstract view of the datapath for a MIPS implementation.



(a) For all the main hardware blocks:

- State their purpose in the datapath.
- Describe their inputs and outputs.
- Explain how they interact with other components.

(b) Why are multiplexers needed in the datapath? Provide the modified datapath after adding the multiplexers.

(c) In maximum two sentences, describe the role of control signals in coordinating data movement between the components. Provide the modified diagram after adding a control unit and its connections to the other components.

[10+5+5=20]

3. Give the MIPS assembly code for the following function:

```
function f(int m, int n) {
 int i, arr[m+n];   // assume memory is allocated to some register
 for (i = 0; i < m + n; i++) {
  if (i < m) {
   arr[i] = i;
  }
  else  {
   arr[i] = 2*i;
  }
 }
 return i;
}
```

Clearly, write your assumptions, if you have any.

See Figure 1 in Appendix-A in page 4 for references. [20]

Part-C

4. (a) Give a diagram of memory types and hierarchy. Briefly describe them in terms of cost and efficiency.
   (b) When you write a code in high level language, how is it converted before executing in hardware. Give each phases and type of software which does them.

[5+5=10]

5. A program runs in 20 seconds on Computer A with a 2.5 GHz clock. A new Computer B must run the same program in 10 seconds. Due to design changes, Computer B requires 1.25 times the number of clock cycles compared to Computer A. What clock rate should Computer B target?

[10]

6. Suppose you have four integers a, b, c, d, each less than 2^{16}, so their upper 16 bits are zero. Assuming you are allowed to use a 32-bit adder **only once**, but you may use basic bitwise and logical operators, Compute $e = a + b$ and $f = c + d$, and store the results in variables e and f. Either write the logic or provide pseudocode or C code.

[10]

7. (a) Consider 8 bit signed integers stored and added in 2's complement form. Is it possible to get a number whose addition with itself is 0? If yes, give an example. If not, give a proof.
   (b) Why guard bit is important, in the context of floating point? Give an example where guard-bit is useful.

[5+5=10]

8. (a) Consider the following code written in C. Assume each word is a 32-bit integer.

```
for (I = 0; I < 8; I++)
for (J = 0; J < 8000; J++)
A[I][J] = B[I][0] + A[J][I];
```

   i. How many 32-bit integers can be stored in a 16-byte cache block?
   ii. Which variable references exhibit temporal locality, why?
   iii. Which variable references exhibit spatial locality, why?

   (b) What is a page fault? Describe the steps performed by the operating system when a page fault occurs.

[5+5=10]

9. Describe the three types of block placement schemes used in cache memory. For each scheme, explain when it performs best and when it performs worst. Also, state which type is most commonly used in modern personal computers and why.

[3+3+4 = 10]

Appendix A

MIPS operands

Name	Example	Comments
32 registers	$s0-$s7, $t0-$t9, $zero, $a0-$a3, $v0-$v1, $gp, $fp, $sp, $ra, $at	Fast locations for data. In MIPS, data must be in registers to perform arithmetic, register $zero always equals 0, and register $at is reserved by the assembler to handle large constants.
2^{30} memory words	Memory[0], Memory[4], . . . , Memory[4294967292]	Accessed only by data transfer instructions. MIPS uses byte addresses, so sequential word addresses differ by 4. Memory holds data structures, arrays, and spilled registers.

MIPS assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add $s1,$s2,$s3	$s1 = $s2 + $s3	Three register operands
	subtract	sub $s1,$s2,$s3	$s1 = $s2 – $s3	Three register operands
	add immediate	addi $s1,$s2,20	$s1 = $s2 + 20	Used to add constants
Data transfer	load word	lw $s1,20($s2)	$s1 = Memory[$s2 + 20]	Word from memory to register
	store word	sw $s1,20($s2)	Memory[$s2 + 20] = $s1	Word from register to memory
	load half	lh $s1,20($s2)	$s1 = Memory[$s2 + 20]	Halfword memory to register
	load half unsigned	lhu $s1,20($s2)	$s1 = Memory[$s2 + 20]	Halfword memory to register
	store half	sh $s1,20($s2)	Memory[$s2 + 20] = $s1	Halfword register to memory
	load byte	lb $s1,20($s2)	$s1 = Memory[$s2 + 20]	Byte from memory to register
	load byte unsigned	lbu $s1,20($s2)	$s1 = Memory[$s2 + 20]	Byte from memory to register
	store byte	sb $s1,20($s2)	Memory[$s2 + 20] = $s1	Byte from register to memory
	load linked word	ll $s1,20($s2)	$s1 = Memory[$s2 + 20]	Load word as 1st half of atomic swap
	store condition. word	sc $s1,20($s2)	Memory[$s2+20]=$s1;$s1=0 or 1	Store word as 2nd half of atomic swap
	load upper immed.	lui $s1,20	$s1 = 20 * 2^{16}	Loads constant in upper 16 bits
Logical	and	and $s1,$s2,$s3	$s1 = $s2 & $s3	Three reg. operands; bit-by-bit AND
	or	or $s1,$s2,$s3	$s1 = $s2 | $s3	Three reg. operands; bit-by-bit OR
	nor	nor $s1,$s2,$s3	$s1 = ~ ($s2 | $s3)	Three reg. operands; bit-by-bit NOR
	and immediate	andi $s1,$s2,20	$s1 = $s2 & 20	Bit-by-bit AND reg with constant
	or immediate	ori $s1,$s2,20	$s1 = $s2 | 20	Bit-by-bit OR reg with constant
	shift left logical	sll $s1,$s2,10	$s1 = $s2 << 10	Shift left by constant
	shift right logical	srl $s1,$s2,10	$s1 = $s2 >> 10	Shift right by constant
Conditional branch	branch on equal	beq $s1,$s2,25	if ($s1 == $s2) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	bne $s1,$s2,25	if ($s1!= $s2) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	slt $s1,$s2,$s3	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than; for beq, bne
	set on less than unsigned	sltu $s1,$s2,$s3	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than unsigned
	set less than immediate	slti $s1,$s2,20	if ($s2 < 20) $s1 = 1; else $s1 = 0	Compare less than constant
	set less than immediate unsigned	sltiu $s1,$s2,20	if ($s2 < 20) $s1 = 1; else $s1 = 0	Compare less than constant unsigned
Unconditional jump	jump	j 2500	go to 10000	Jump to target address
	jump register	jr $ra	go to $ra	For switch, procedure return
	jump and link	jal 2500	$ra = PC + 4; go to 10000	For procedure call

Figure 1: Information as per the MIPS 32-bit architecture. [from course textbook]

. . . . . x . . . . . .